

UNIDAD N° 1: INGENIERÍA DE SW EN CONTEXTO

CRISIS DEL SOFTWARE

La crisis del software es un término que surgió en 1968, durante una conferencia de la OTAN, para describir las dificultades que enfrentaba el desarrollo de software debido a la alta demanda y a la complejidad creciente de los proyectos. Estos problemas provocaban altos costos, retrasos y software de baja calidad, lo que evidenció la necesidad de nuevas metodologías y herramientas para mejorar el desarrollo. En esa misma conferencia surgió el término de Ingeniera de Software, para intentar dar solución a la Crisis del Software.

A pesar de los avances, la crisis del software no ha desaparecido, aunque hoy se gestiona mejor gracias a metodologías, buenas prácticas y herramientas modernas. Esto se debe a las propiedades esenciales del software: complejidad, invisibilidad, conformidad y capacidad de cambio, que hacen que siempre existan desafíos en su desarrollo.

UN POCO DE HISTORIA



CMM --> modelo de madurez y capacidad. Es un convenio de buenas prácticas.

CONCEPTO DE INGENIERÍA DE SOFTWARE

Es una disciplina que se interesa por todos los aspectos del desarrollo de software, desde la especificación del sistema hasta el mantenimiento del mismo después de su puesta en operación.

La importancia de la Ingeniera de Software radica en dos razones:

- Las personas dependen mucho de los sistemas de software para la vida cotidiana, por lo que es necesario crear sistemas de software de manera rápida y confiable.
- Aplicar métodos y técnicas de Ingeniería de Software hace que sea más fácil de mantener y modificar los sistemas.

DISCIPLINAS DE LA ING DE SOFTWARE

1. Disciplinas técnicas → Son las actividades directamente relacionadas con la creación del producto de software.

- Requerimientos
- Análisis
- Diseño
- Construcción
- Prueba
- Despliegue

2. Disciplinas de gestión → Son las actividades orientadas a organizar, planificar y controlar el desarrollo del proyecto de software.

- Planificación de Proyecto
- Monitoreo y Control de Proyectos

3. Disciplinas de soporte → Son las actividades que aseguran la calidad y el control durante todo el ciclo de vida del software.

- Gestión de Configuración de Software
- Aseguramiento de Calidad
- Métricas

PROYECTOS DE SOFTWARE FALLIDOS Y EXITOSOS

➤ Cuando los proyectos son exitosos, se debe a que:

- Involucran al usuario (15.9%)
- Reciben apoyo de la gerencia (13%) → la dirección de la empresa respalda el proyecto
- Hay un enunciado claro de los requerimientos (9.6%)
- Realizan un planeamiento adecuado (8.2%)
- Expectativas realistas (7.7%)
- Hitos intermedios (7.7%)
- Personas involucradas competentes (72%)

Ejemplo de proyectos exitosos: WhatsApp, Netflix, GitHub

➤ Cuando los proyectos fallan se debe a lo siguiente:

- Requerimientos incompletos (13.1%)
- Falta de involucramiento del usuario (12.4 %)
- Falta de recursos (10.6 %)
- Expectativas poco realistas (9.3 %)
- Falta de apoyo de la Gerencia (8.7 %)
- Requerimientos cambiantes (8.1 %)

Ejemplos de proyectos fallidos:

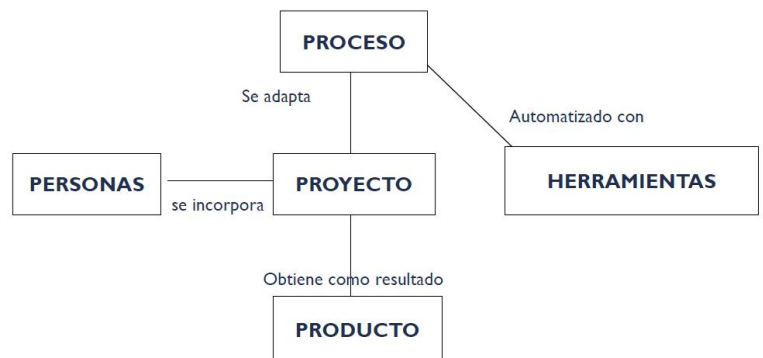
PRINCIPALES PROBLEMAS CON EL DESARROLLO DE SOFTWARE

- La versión final del producto no satisface las necesidades del cliente.
- No es fácil extenderlo y/o adaptarlo. Agregar más funcional en otra versión es casi una misión imposible
- Mala documentación
- Mala calidad
- Más tiempos y costos que los presupuestados

COMPONENTES DE PROYECTO DE DESARROLLO DE SOFTWARE

En un proyecto de desarrollo de software existen tres elementos clave: proceso, proyecto y producto, que se relacionan entre sí de la siguiente manera:

El **proceso** define cómo se va a desarrollar el trabajo mediante métodos, prácticas y actividades, y se adapta a las características del proyecto. El **proyecto** aplica ese proceso para planificar, organizar y controlar todas las actividades necesarias, asegurando que se cumplan los objetivos planteados del proyecto. Finalmente, se obtiene el **producto** como resultado de aplicar el proceso dentro del proyecto. Este producto debe cumplir con los requisitos y objetivos definidos.



PROCESOS

Un proceso de software es un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

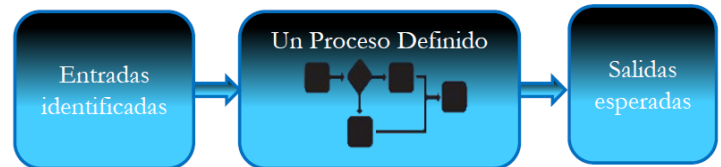
Los procesos de software son una guía general de actividades, pero no se aplican siempre de la misma forma: se adaptan al proyecto según su tamaño, complejidad, presupuesto y equipo.

PROCESOS EMPÍRICOS Y DEFINIDOS

- Procesos definidos

Son procesos estructurados y planificados, con pasos claros y consistentes que se siguen siempre de la misma manera. Esto permite que distintos equipos los repitan y obtengan resultados similares, lo que facilita la organización, el control y reduce la improvisación. Además, al estar bien especificados, permiten predecir con mayor confiabilidad el tiempo, el esfuerzo y la calidad del software.

Un ejemplo es el PUD.



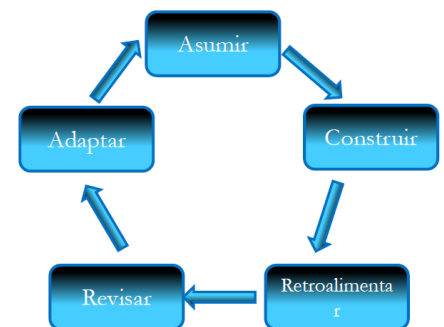
- Proceso empíricos

Son procesos que no tienen pasos fijos ni resultados totalmente predecibles, ya que dependen del contexto y de la experiencia de las personas que los ejecutan. Se gestionan mediante ciclos cortos de inspección y adaptación, lo que permite ajustar el trabajo continuamente en proyectos complejos o cambiantes.

Estos procesos se basan en un ciclo de aprendizaje:

1. Asumir → Se parte de una hipótesis o suposición.
2. Construir → Se lleva esa idea a la práctica, creando una primera versión o solución.
3. Retroalimentar → Se recopilan comentarios y resultados de lo construido.
4. Revisar → Se analiza la información obtenida: qué funcionó, qué no funcionó, qué problemas aparecieron.
5. Adaptar → Se ajusta el proceso o la solución en base a la revisión y la retroalimentación.
6. Asumir → Con lo aprendido, se generan nuevas hipótesis y el ciclo vuelve a empezar.

Ejemplo: Scrum.



PROYECTO

Un proyecto es una unidad de gestión de trabajo en el que se organiza a las personas y se gestionan los recursos para poder obtener como resultado el producto de software.

CARACTERÍSTICAS DE UN PROYECTO

- Resultado único → Cada proyecto, aunque sea similar a otros, tiene características que lo hacen único.
- Orientados a objetivos → Los proyectos están dirigidos a obtener resultados y ello se refleja a través de objetivos. Los objetivos deben ser alcanzables y no ambiguos.
- Tiene una fecha de inicio y una fecha de fin → Los proyectos son temporarios, cuando se alcanza el/los objetivo/s, el proyecto termina.
- Elaboración gradual → El proyecto se desarrolla paso a paso, ajustando y refinando el producto a medida que se avanza.
- Tareas interrelacionadas → Las actividades del proyecto están conectadas entre sí, de manera que el avance de unas depende del progreso de otras.

ADMINISTRACIÓN DE UN PROYECTO (PROJECT MANAGMENT)

Es la disciplina que tiene como función principal organizar y coordinar los recursos y las personas involucradas en un proyecto, con el fin de cumplir los objetivos del proyecto en tiempo, forma y dentro del presupuesto. Implica **planificar y luego monitorear** constantemente el avance del proyecto, comparando lo real con lo planificado para detectar desvíos y hacer los ajustes necesarios.

Requiere conocimientos técnicos específicos, ya que no basta con dirigir personas: es necesario aplicar conocimientos, habilidades, herramientas y técnicas que permitan cumplir con los requerimientos establecidos del proyecto.

Incluye las siguientes actividades:

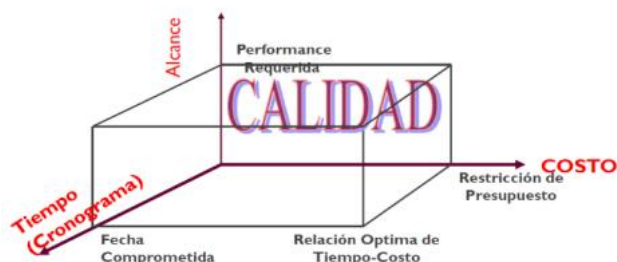
- Identificar los requerimientos.
- Establecer objetivos claros y alcanzables.
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders)

Administrar un proyecto no asegura el éxito, pero no hacerlo casi garantiza el fracaso, porque se pierde control sobre el tiempo, el costo y la calidad.

LA TRIPLE RESTRICCIÓN

La triple restricción establece que todo proyecto está condicionado por tres variables: alcance, costo y tiempo.

- Alcance: qué se va a entregar al cliente, es decir, los requerimientos y objetivos del proyecto.
- Costo: los recursos necesarios para cumplir con el alcance.
- Tiempo: el plazo en el que se entregarán los resultados.



El cliente es quien define el alcance, y a partir de este se determinan el tiempo y el costo. Cuando estas variables se desequilibran, se compromete la calidad del producto, la cual no es negociable. Por eso, el rol del líder de proyecto es equilibrarlas, negociando con el cliente. Por ejemplo, si el cliente exige un alcance amplio, el líder debe aclarar que eso requiere más tiempo y más presupuesto. En cambio, si el cliente quiere reducir tiempos y costos, el líder debe ajustar el alcance para que sea realista dentro de esas limitaciones.

En el enfoque tradicional, el alcance del proyecto se define al inicio y no cambia durante el desarrollo. Cada fase o iteración termina únicamente cuando se han completado todas las tareas planificadas dentro de ese alcance.

LÍDER DE PROYECTO / EQUIPO

Un proyecto necesita gente, que es la que se integra. El software es una actividad intelectual, por lo que no se puede agregar personas en cualquier momento, ya que el proyecto se atrasa ya que se debe capacitar al personal.

El trabajo que la gente hace es un recurso del proyecto, pero la gente no es recurso.

LÍDER DE PROYECTO

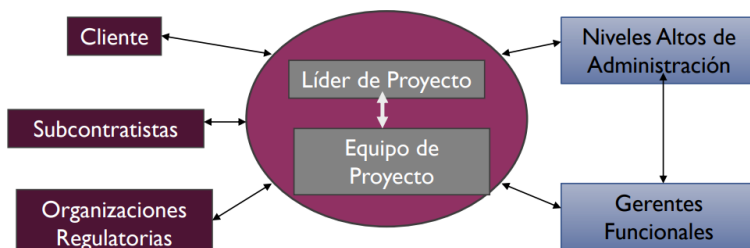
El líder de proyecto es la persona responsable de planificar, coordinar y supervisar todas las actividades del proyecto para que se cumplan los objetivos, tiempo y costo. Su función es organizar al equipo, comunicar con los stakeholders, controlar el avance, gestionar riesgos y balancear la triple restricción, asegurando que el producto final se entregue con la calidad esperada.

EQUIPO DE PROYECTO

Un equipo de proyecto es un grupo multidisciplinario, donde cada integrante cumple un rol específico y aporta sus conocimientos y habilidades para alcanzar los objetivos planteados. Sus miembros trabajan de forma coordinada, se sienten responsables no solo de sus tareas individuales, sino también del resultado colectivo.

Características de un equipo de proyecto:

- Diversos conocimientos y habilidades
- Posibilidad de trabajar juntos efectivamente / desarrollar sinergia
- Usualmente es un grupo pequeño
- Tienen sentido de responsabilidad como una unidad



PLANIFICACIÓN DEL PROYECTO

Planificar un proyecto es como tener una hoja de ruta para un viaje: define qué se va a hacer, cómo, cuándo y quién será responsable. El plan del proyecto es el documento que guía al equipo y permite organizar el trabajo.

1. DEFINIR EL OBJETIVO DEL PROYECTO

Se debe establecer con claridad qué se busca lograr con el proyecto y cuál es su propósito principal. Los objetivos deben ser claros, específicos y alcanzables, es decir, que puedan cumplirse con un esfuerzo razonable y dentro de un tiempo definido. El objetivo es la guía que orienta todas las actividades del proyecto y permite evaluar si se alcanzó el éxito al finalizarlo.

2. DEFINIR EL ALCANCE DEL PROYECTO

El alcance del proyecto establece todo el trabajo necesario para cumplir con los objetivos y entregar el producto o servicio con las características y funciones especificadas.

Es importante diferenciarlo del alcance del producto:

- **Alcance del producto:** define qué funcionalidades debe tener el software para satisfacer los requerimientos del cliente (medido contra la Especificación de Requerimientos, ERS).
- **Alcance del proyecto:** define todas las actividades y tareas necesarias para desarrollar y entregar el producto (medido contra el Plan de Proyecto).

3. DEFINIR EL PROCESO Y CICLO DE VIDA

Definir el proceso y el ciclo de vida de un proyecto implica responder a la pregunta: “¿Cómo se va a desarrollar el proyecto?”. El proceso establece la forma en la que se realizarán las tareas y actividades que se van a realizar en el proyecto. Elegir el ciclo de vida indica cómo y en qué momento se harán esas tareas, así como la cantidad de esfuerzo que se dedicará a cada una en cada etapa.

En conjunto, ambos determinan cómo se llevará adelante el proyecto desde su inicio hasta la entrega del producto final.

4. DEFINIR EL EQUIPO DEL PROYECTO

Esto implica identificar qué perfiles se necesitan (analistas, diseñadores, desarrolladores, testers, gestores, etc.), seleccionar a los integrantes adecuados y definir su rol y asignarle responsabilidades.

Se debe hacer especial foco en la selección del personal, ya que las capacidades y habilidades del equipo impactan directamente en la calidad y en el correcto desarrollo del proyecto.

5. ESTIMAR

La estimación es un proceso probabilístico que busca predecir aspectos del proyecto antes de que ocurran, como tamaño, esfuerzo, tiempo y costo. Es importante entender que estas estimaciones se realizan con información limitada y siempre tienen un grado de incertidumbre.

En el enfoque tradicional se estima las siguientes cosas y en este orden:

- **Tamaño** → se estima sobre el producto de software a construir, utilizando los requerimientos para determinar qué tanto trabajo implicará.
- **Esfuerzo** → derivado del tamaño, indica cuántas horas de trabajo serán necesarias por persona. El esfuerzo aumenta proporcionalmente al tamaño del proyecto.
- **Calendario** → basado en el esfuerzo estimado, se calcula cuándo se podrán completar las tareas, considerando también la disponibilidad de personal y la posibilidad de hacer tareas en paralelo.
- **Costo** → depende directamente del esfuerzo y del tamaño del proyecto. Permite dar al cliente una idea aproximada del presupuesto necesario.
- **Recursos críticos** → depende directamente del esfuerzo y del tamaño del proyecto. Permite dar al cliente una idea aproximada del presupuesto necesario.

6. PROGRAMACIÓN DE PROYECTOS (CALENDARIZACIÓN)

La programación de proyectos consiste en detallar todas las tareas necesarias para completar el proyecto, tomando como referencia la estimación de calendario previa.

Esto implica dividir el proceso de desarrollo en actividades pequeñas y detalladas, identificando qué se hará, quién lo hará, cuándo y cuánto esfuerzo requiere. Para organizar y visualizar esta planificación, se suelen usar herramientas como el Diagrama de Gantt,

7. GESTIÓN DE RIESGOS

El riesgo es la probabilidad de que ocurra un evento que afecte el cumplimiento del proyecto. Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo, la cual se calcula multiplicando las siguientes dos variables.

- Probabilidad → qué tan probable es que ocurra el riesgo
- Impacto → qué tan grave sería el efecto sobre el proyecto

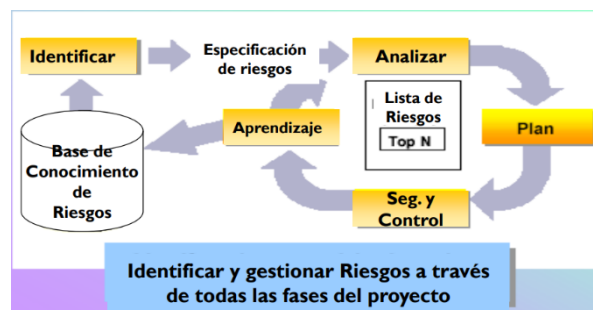
Esta exposición nos ayuda a definir que riesgos se deben resolver primero.

ACTITUDES FRENTE A LOS RIESGOS

- Negación: hacer de cuenta que los riesgos no existen.
- Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
- Gestión Proactiva: ir trabajando sobre el riesgo generado planes de mitigación y de contingencia

PASOS PARA GESTIONAR RIESGOS

1. Identificar el riesgo: detectar posibles eventos que puedan afectar el proyecto.
2. Analizar el riesgo: evaluar probabilidad, impacto y exposición.
3. Realizar un plan: definir estrategias de mitigación, contingencia o aceptación.
4. Seguridad y control: asegurarse de que las acciones se ejecuten y los riesgos se mantengan bajo control.



8. DEFINICIÓN DE MÉTRICAS

Las métricas son medidas numéricas que permiten visualizar y evaluar el avance, estado o desempeño de un proyecto, proceso o producto en un momento determinado. Deben ser simples y medibles, para que realmente aporten información útil y clara sobre el proyecto.

En proyectos de software, las métricas más comunes son:

- Tamaño del producto
- Esfuerzo
- Calendario (tiempo)
- Defectos

⚠ Las métricas no se deben usar para medir a las personas ni para premios o castigos, porque eso genera que los integrantes oculten información o mientan sobre su trabajo.

9. DEFINICIÓN DE CONTROLES

Los controles se basan en las métricas definidas y permiten verificar que el proyecto se está desarrollando según lo planificado. Su objetivo es monitorear y asegurar la calidad, detectar desviaciones en tiempo, costo, alcance o esfuerzo, y tomar acciones correctivas cuando sea necesario.

MONITOREO Y CONTROL DEL PROYECTO

El monitoreo y control consiste en verificar día a día cómo avanza el proyecto y compararlo con lo que se había planificado. Permite detectar desviaciones a tiempo y tomar acciones correctivas cuando sea necesario.

Su objetivo es dar visibilidad del estado del proyecto, estableciendo mecanismos de comunicación para informar a todos los involucrados sobre lo que está sucediendo y garantizar que el proyecto avance según lo planeado.

CICLO DE VIDA O MODELO DE PROCESOS DE UN PROYECTO

El ciclo de vida define cómo se organiza el desarrollo del proyecto en fases y el orden en el cual se llevan a cabo, indicando en qué momento se deben realizar las actividades. Por su parte, el proceso establece qué actividades se van a realizar.

En otras palabras, el ciclo de vida implementa y organiza las actividades definidas por el proceso a lo largo del proyecto.

Hay 3 tipos básicos de ciclos de vida:

1. **Secuencial** → Las fases se realizan de manera lineal, una después de otra. Solo se pasa a la siguiente fase cuando la anterior está completamente terminada.

Ventajas:

- Tienen una estructura clara y fácil de entender.
- Útil en proyectos con requisitos bien definidos desde el inicio.

Desventajas:

- Es poco flexible ante cambios; difícil adaptarse si los requisitos cambian.
- Detectar errores tarde puede ser costoso.
- No permite entregar resultados parciales al cliente durante el desarrollo.

Ejemplo: Modelo Cascada, Modelo en V

2. **Iterativo** → El desarrollo se realiza en pequeñas iteraciones o ciclos, donde se construye una versión parcial del software, se prueba y se mejora en la siguiente iteración.

Ventajas:

- Permite ajustes frecuentes según retroalimentación del cliente.
- Los errores se detectan y corrigen más rápido.
- Se pueden entregar versiones parciales o prototipos antes del producto final.
- Muy útil para sistemas de requerimientos cambiantes.

Desventajas:

- Requiere buena planificación de cada iteración.
- Puede volverse costoso si no se controlan las iteraciones.
- Alto costo de documentar cada incremento.

Ejemplo: Iterativo e incremental

3. **Recursivo** → El proyecto se desarrolla iterativamente dentro del equipo, refinando internamente el producto en cada ciclo. No se entregan versiones parciales al cliente; el objetivo es mejorar y consolidar el producto internamente hasta la versión final.

Ventajas:

- Integra análisis de riesgos en cada fase.
- Se mejora continuamente el sistema y se minimizan errores.
- Flexible ante cambios y ajustes complejos.

Desventajas:

- Más complejo de gestionar.
- Requiere experiencia del equipo.
- Puede ser costoso en tiempo y recursos.

INFLUENCIA EN LA ADMINISTRACIÓN DE PROYECTOS DE SOFTWARE

El tipo de ciclo de vida elegido afecta directamente cómo se administra el proyecto porque determina:

- Planificación: cómo se estiman tiempos, recursos y costos.
- Control: cómo se monitorea el avance y se gestionan los riesgos.

- Asignación de tareas: cómo se distribuyen las responsabilidades entre los miembros del equipo.
- Puntos de revisión: cuándo se evalúa el progreso y se toman decisiones de ajuste.

En pocas palabras, el modelo de proceso elegido guía toda la gestión del proyecto, afectando cómo se planifica, organiza, controla y entrega el software.

CRITERIOS PARA ELEGIR UN CICLO DE VIDA DE PROYECTO

La elección del ciclo de vida (secundario, iterativo o recursivo) depende de las necesidades del proyecto y de las características del producto. Los principales criterios son:

- Claridad de los requisitos:
 - Si los requisitos están claros y estables, se puede usar un ciclo secuencial, porque no se esperan muchos cambios.
 - Si los requisitos son incompletos o inciertos, conviene un ciclo iterativo o recursivo, que permite adaptar el desarrollo sobre la marcha.
- Complejidad del producto:
 - Proyectos grandes y complejos pueden necesitar un enfoque incremental o iterativo para manejar mejor las fases y los riesgos.
- Necesidad de retroalimentación del cliente:
 - Si el cliente quiere ver resultados parciales y dar retroalimentación, conviene un ciclo iterativo.
 - Si el cliente solo necesita el producto final, el recursivo puede ser suficiente.
- Recursos y tiempo disponibles:
 - Si el proyecto tiene plazos estrictos y recursos limitados, un ciclo secuencial puede ser más fácil de controlar.
 - Si hay flexibilidad y el equipo es experimentado, los ciclos iterativos y recursivos permiten aprender y mejorar durante el desarrollo.
- Riesgos asociados al proyecto:
 - Proyectos con alta incertidumbre o riesgo técnico se benefician de ciclos que permitan pruebas, ajustes y refinamiento continuo (iterativo o recursivo).
 - Proyectos de bajo riesgo pueden manejarse con ciclos secuenciales.

UNIDAD N°2: GESTIÓN LEAN – ÁGIL DE PRODUCTOS DE SOFTWARE

MANIFIESTO ÁGIL

El Manifiesto Ágil es un documento creado en 2001 por un grupo de expertos en desarrollo de software, donde se establecen los principios y valores que guían la forma de trabajar con metodologías ágiles. Su objetivo es proponer una manera más flexible, colaborativa y eficiente de gestionar proyectos de software.

VALORES DEL MANIFIESTO ÁGIL

1. **Individuos e interacciones, por sobre procesos y herramientas:** Esto significa que lo más importante son las personas y cómo se comunican, porque ellas son las que generan soluciones y se adaptan a los cambios. Los procesos y herramientas ayudan, pero no consiguen resultados por sí solas.
2. **Software funcionando por sobre documentación extensiva:** Lo prioritario es entregar un producto que funcione y dé valor al cliente, más que generar grandes cantidades de documentación. La documentación no desaparece, pero se busca que sea la justa y necesaria.
3. **Colaboración con el cliente por sobre negociación contractual:** En lugar de enfocarse solo en cumplir un contrato rígido, se busca trabajar en conjunto con el cliente, escuchando sus necesidades y ajustando el producto según lo que realmente requiere. La colaboración continua con el cliente genera más valor que el cumplimiento estricto de un contrato.
4. **Responder a cambios por sobre seguir un plan:** Los planes son útiles, pero en un entorno cambiante es más importante poder adaptarse rápidamente a los cambios que seguir un plan fijo. Las metodologías ágiles promueven la anticipación y la adaptación, frente a la planificación y el control que proponen las fórmulas de gestión tradicionales.

PRINCIPIOS

1. Satisfacer al cliente
2. Aceptar que los requisitos cambian
3. Entregar software funcional frecuentemente
4. Los responsables de negocios, diseñadores y desarrolladores deben trabajar juntos día a día durante el proyecto
5. Desarrollar proyectos en torno a individuos motivados:
6. Comunicar información cara a cara
7. El software funcionando es la principal medida de éxito
8. Los procesos ágiles promueven el desarrollo sostenible
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad
10. La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.
11. Las mejores arquitecturas, diseños y requerimientos surgen de equipos auto-organizados
12. A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto ajustan su comportamiento

FILOSOFÍA ÁGIL

La filosofía ágil es una manera de gestionar proyectos y desarrollar software que surge como respuesta a los métodos tradicionales, que eran muy rígidos y poco adaptables. Su idea central es que en un mundo cambiante no se puede planificar todo de antemano, por lo que se necesita un enfoque flexible que permita adaptarse a los cambios y entregar valor al cliente de manera continua. La filosofía ágil se materializa en el manifiesto ágil.

La filosofía ágil se apoya en procesos empíricos, es decir, procesos basados en la experiencia, la retroalimentación y el aprendizaje continuo. Estos procesos funcionan mediante ciclos de vida iterativos, donde en cada iteración se inspecciona lo realizado, se reciben aprendizajes y se adaptan los próximos pasos.

PILARES DEL EMPIRISMO

- **Transparencia:** Toda la información relevante del proyecto debe ser visible para aquellos que realizan el trabajo, como para los que reciben el trabajo. Permite que todos comprendan en qué estado está el proyecto, qué se está haciendo y qué problemas existen.

La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

- **Inspección:** Consiste en revisar periódicamente el progreso del trabajo y el cumplimiento de los objetivos. Sirve para detectar desviaciones, errores o mejoras necesarias lo antes posible.

La inspección permite la adaptación. La inspección sin adaptación se considera inútil.

- **Adaptación:** Cuando la inspección revela un problema o una oportunidad de mejora, el equipo debe ajustar el plan, proceso o producto. Esto garantiza que el proyecto se mantenga alineado con las necesidades reales.

En cada ciclo de inspección-adaptación hay aprendizaje.

DESARROLLO ÁGIL DE SOFTWARE

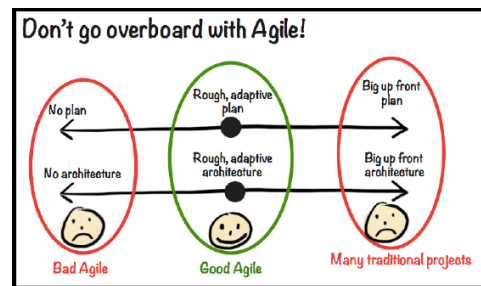
Martin Fowler dijo que Agile es: “Un compromiso útil entre nada de proceso y demasiado proceso”

Con esta frase se quiere decir que Agile representa un punto intermedio entre no tener proceso y tener demasiado proceso. Sin proceso, el trabajo se vuelve caótico y desorganizado, mientras que con un exceso de procesos aparece la burocracia, con reglas rígidas y documentación excesiva que dificultan la adaptación.

Ágil busca el equilibrio: aplicar solo los procesos necesarios para organizar el trabajo y asegurar calidad, pero sin frenar la flexibilidad del equipo. De esta manera se logra un desarrollo más eficiente, adaptable a cambios y enfocado en entregar valor al cliente de forma continua.

¿QUÉ ES SER ÁGIL?

Ser ágil significa adoptar la filosofía ágil, es decir, seguir un conjunto de valores y principios que guían la forma de trabajar en el desarrollo de un producto. No es una metodología ni un proceso específico, sino una mentalidad de flexibilidad, colaboración y adaptación al cambio. Ser ágil no implica trabajar sin disciplina; al contrario, requiere organización, compromiso y responsabilidad para entregar valor de manera continua y efectiva.



FRAMEWORKS ÁGILES

En el desarrollo ágil de software, se utilizan frameworks ágiles. Un framework ágil es un marco de trabajo que traduce los valores y principios de la filosofía Ágil en una forma práctica de organizar el desarrollo de software. Algunos ejemplos son: Scrum, FDD, ATDD, Crystal, XP

REQUERIMIENTOS ÁGILES

En Agile, los requerimientos se gestionan de forma distinta al enfoque tradicional:

- En lugar de definir una lista completa de requerimientos desde el inicio, en Agile se parte de lo mínimo necesario para comenzar el proyecto. Se arranca con una visión incompleta del producto, priorizando los requerimientos que generan más valor para el negocio o los usuarios, con el objetivo de construir el producto correcto. Luego, en cada iteración, los requerimientos se van agregando, ajustando o cambiando según lo que se aprende y lo que se necesita.
- Para alinear a todos los involucrados, se utilizan herramientas como historias de usuario, prototipos u otras técnicas, que permiten compartir una visión clara del producto sin necesidad de especificar todos los detalles desde el comienzo.

PRINCIPIOS ÁGILES RELACIONADOS

- **Principio 1: Satisfacer al cliente** → Los requisitos ágiles se priorizan según el valor de negocio.
- **Principio 2: Aceptar que los requisitos cambian** → En Agile no se definen todos los requisitos al inicio. Se empieza con lo mínimo y se admite el cambio constante, porque el producto evoluciona con las necesidades.
- **Principio 4** → Esta interacción constante asegura que los requerimientos representen correctamente el valor de negocio y que el producto se construya en línea con lo que realmente se necesita.

- **Principio 6: Comunicar información cara a cara** → Los requerimientos ágiles se construyen y ajustan principalmente mediante la comunicación directa y continua, en lugar de depender de documentación extensa.
- **Principio 11** → Los mejores requerimientos surgen de equipos auto-organizados porque al trabajar juntos y con libertad de decisión, combinan la visión del negocio con la experiencia técnica, logrando definiciones más útiles y de mayor calidad.

BRUF, PRINCIPIO DE PARETO Y ENFOQUE JUST IN TIME

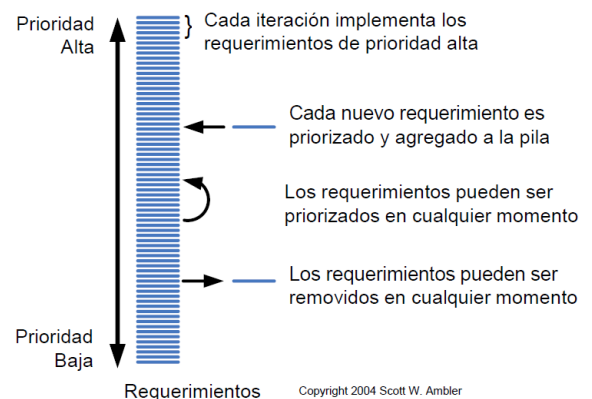
En Agile se evita el enfoque **BRUF (Big Requirements Up Front)**, que en métodos tradicionales intenta definir todos los requerimientos desde el inicio. Se lo evita porque los requerimientos cambian constantemente y es imposible preverlos por completo. Esto se complementa con el **Principio de Pareto (80/20)**, que indica que aproximadamente el 20% de las funcionalidades generan el 80% del valor del producto. Por lo tanto, el equipo se enfoca primero en las funcionalidades que aportan mayor impacto al negocio y a los usuarios.

Para facilitar esto, Agile aplica el enfoque **Just in Time**, identificando y especificando los requerimientos solo cuando son necesarios, en lugar de invertir tiempo en definirlos todos al inicio. Esto reduce el riesgo de trabajar en elementos que luego cambien o dejen de ser importantes, y asegura que el desarrollo se concentre en entregar valor real de manera continua.

PRODUCT OWNER Y PRODUCT BACKLOG

El Product Owner (PO) es la persona responsable de definir el producto. Su rol no es técnico, sino de negocio, ya que debe garantizar que el desarrollo esté alineado con el valor que el producto debe aportar a los usuarios y a la organización. Para cumplir esta función, el PO utiliza el Product Backlog, que es una lista priorizada de características o requerimientos que se espera que el producto satisfaga.

A diferencia del enfoque tradicional (donde todos los requerimientos se documentaban juntos en una ERS antes de empezar), en el agílísimo el Product Backlog no necesita estar completo para iniciar el trabajo. Es un artefacto dinámico, que se va actualizando a medida que avanza el proyecto (los requerimientos que se resuelven se sacan del product backlog)



TIPOS DE REQUERIMIENTOS

- **Requerimiento de negocio** → Expresan el objetivo o necesidad del negocio que justifica el proyecto.
- **Requerimiento de Usuario** → Reflejan lo que los usuarios finales necesitan hacer con el sistema.
- **Requerimiento Funcional**
- **Requerimiento No funcional**
- **Requerimiento de Implementación**

Estos requerimientos trabajan en el dominio del problema, a diferencia de los requerimientos de software (funcionales y no funcionales) que trabajan en el dominio de la solución.



IDEAS CLAVES DEL ENFOQUE ÁGIL SOBRE LOS REQUERIMIENTOS

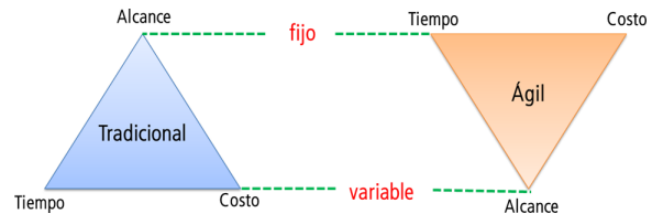
- “Los cambios son la única constante” → los requerimientos y necesidades de los usuarios cambian constantemente, ya sea por nuevas prioridades, descubrimientos durante el desarrollo o cambios en el mercado.
- “Stakeholders: no son todos los que están” → No todos los involucrados en un proyecto conocen realmente lo que se necesita o tienen autoridad para decidir. Hay que identificar quiénes son las personas que realmente pueden aportar valor o tomar decisiones.
- “El usuario dice lo que quiere cuando recibe lo que pidió” → Muchas veces los usuarios no saben exactamente lo que necesitan hasta que ven un prototipo o una versión inicial; por eso se trabaja de manera iterativa y con feedback constante.
- “No hay técnicas ni herramientas que sirvan para todos los casos” → Cada proyecto es distinto, por lo que los métodos, herramientas o plantillas deben adaptarse al contexto y al equipo.
- “Lo importante no es entregar un requerimiento, sino un resultado de valor”

TRIPLE RESTRICCIÓN

En Agile, la triple restricción (alcance, tiempo y costo) se maneja de forma diferente al enfoque tradicional:

- En lugar de tener alcance fijo, Agile mantiene tiempo y recursos fijos. Cada iteración (o sprint) tiene una fecha de inicio y fin establecida, y el equipo trabaja con los recursos asignados.
- El alcance es variable, es decir, se ajusta según lo que se pueda completar dentro de la iteración. Si no se termina todo lo planificado, lo que no se completó se prioriza para la siguiente iteración.

De esta manera, se asegura que el proyecto avance de manera constante y se entregue valor continuamente, en lugar de retrasarse intentando cumplir un alcance rígido.



USER STORIES

Una user story es una descripción breve en lenguaje natural de una funcionalidad o necesidad que un usuario o cliente espera del sistema. Son una técnica utilizada en Agile para gestionar requerimientos y trabajan a nivel de requerimientos de negocio y de usuario.

Las US son escritas generalmente por el Product Owner (PO) y son multipropósito:

- Reflejan una necesidad del usuario.
- Describen una característica del producto.
- Sirven como ítem de planificación.
- Funcionan como un token para iniciar conversaciones con el equipo.
- Permiten diferir discusiones hasta que se tenga más información.

PARTES DE UNA US

- Conversación: Es la comunicación entre el PO y el equipo sobre lo que se espera de la US. Es la parte más importante, porque permite aclarar dudas y entender el valor del requisito.
- Tarjeta: es la parte visible de una US y describe la característica del producto que se espera que el cliente reciba.
- Confirmación: La confirmación de una User Story es la verificación de que se cumplieron sus criterios de aceptación y que la funcionalidad entregada satisface la necesidad del usuario.

ESTRUCTURA DE LA TARJETA

- Frase verbal → es una forma corta de referenciar la US para identificarla de forma rápida. Es opcional.
- Descripción → Como <nombre del rol> quiero <actividad> de forma tal que <valor del negocio que recibo>
- Criterios de aceptación → Definen los límites y condiciones que una US debe cumplir para entregar valor. Ayudan a desarrolladores y testers a derivar las pruebas y saber cuándo detener la adición de funcionalidades. Son de alto nivel, definen la intención y no la solución, y garantizan que el producto construido sea aceptado por el Product Owner.
- Pruebas de aceptación → se diseñan en base a los criterios de aceptación y permiten verificar objetivamente que la historia cumple lo acordado.

PORCIONES VERTICALES

Las US se desarrollan en porciones verticales, lo que significa que cada historia incluye lo necesario para entregar valor completo al usuario:

- Un poco de interfaz
- Un poco de lógica de negocio
- Un poco de base de datos

Esto asegura que cada historia sea entregable y funcional, y no solo un pedazo aislado que no aporta valor.

CRITERIOS PARA ACORDAR COMO EQUIPO

DEFINITION OF READY (DOR)

Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

INVEST MODEL

Es un modelo de calidad que ayuda a evaluar que tan bien escritas están las US.

- Independientes → cada US debe ser autónoma, sin solaparse en concepto con otras, de manera que se pueda programar e implementar en cualquier orden dentro del proyecto.
- Negociables → una US no es un contrato rígido de características, los detalles de la misma se discuten entre el Product Owner y el equipo durante el desarrollo.
- Valiosas → Una US debe entregar valor real al cliente. Su importancia se mide por el beneficio que aporta al usuario final.
- Estimables → Una US debe poder ser estimada por el equipo, al menos de manera suficiente para ayudar a priorizar y planificar su implementación. La capacidad de estimación depende de que la historia sea entendida (negociable), de su tamaño (historias grandes son más difíciles de estimar) y de la experiencia del equipo.
- Pequeñas → La US debe ser lo suficientemente pequeña como para completarse en una iteración.
- Testeables → Una buena US debe poder ser probada para verificar que cumple con lo esperado. Si no se puede probar, puede indicar que la historia no está lo suficientemente clara.

DEFINITION OF DONE (DOD)

Es un acuerdo del equipo que define cuándo se considera que el trabajo asociado a una User Story está terminado. Sirve para garantizar que la US cumpla todos los criterios de aceptación y esté lista para entregarse.

DIFERENTES NIVELES DE ABSTRACCIÓN DE LAS US

Cuando una User Story es demasiado grande para completarse dentro de un sprint, se la denomina Épica. Si es necesario agrupar varias User Stories y/o épicas bajo un objetivo común o línea de investigación, se utiliza el concepto de Tema.

Tanto las épicas como los temas se consideran niveles de granularidad demasiado altos, por lo que no cumplen con el criterio de Definition of Ready (DoR). Por esta razón, suelen ubicarse en la parte inferior del Product Backlog, hasta que se descomponen en historias más pequeñas y manejables.



SPIKES

Un Spike es un tipo especial de User Story que surge cuando existe incertidumbre (técnica o funcional) que genera riesgo y hace que la historia no pueda ser estimada. Para resolver esa incertidumbre, el equipo debe destinar tiempo para realizar la investigación necesaria, esa investigación permite transformar el Spike en una User Story que cumpla con el DOR.

Al igual que las demás historias, los Spikes se registran en el Product Backlog.

¿PARA QUÉ SE UTILIZAN LOS SPIKES?

- Para que el equipo se familiarice con una nueva tecnología o dominio.
- Para analizar una historia compleja y dividirla en piezas más manejables.
- Para reducir riesgos tecnológicos, investigando o creando prototipos que den confianza.
- Para resolver riesgos funcionales, cuando no está claro cómo debe interactuar el usuario con el sistema.

CLASIFICACIÓN

Las Spikes se clasifican en:

- **Técnicas:** se usan cuando el equipo no tiene claro cómo resolver algo a nivel tecnológico. Por ejemplo: probar si una nueva base de datos soporta la cantidad de usuarios que necesitamos, investigar qué framework usar, o validar un enfoque de arquitectura. Sirven para ganar seguridad técnica antes de comprometerse a una funcionalidad
- **Funcionales:** se usan cuando no está claro qué quiere el usuario o cómo debería funcionar algo en la práctica, usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios involucrados.

Algunas User Stories requieren de ambos tipos de spikes.

LINEAMIENTOS DE SPIKE

- Estimables, demostrables, y aceptables
- La excepción, no la regla → Siempre va a haber incertidumbre en los proyectos, y el equipo tiene que aprender a manejarla de forma natural en cada iteración. Los spikes se reservan solo para dudas grandes o críticas, donde no se puede avanzar sin investigar antes. Es como un “último recurso”, no la primera opción.
- Implementar la spike en una iteración separada de las historias resultantes. Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

ESTIMACIONES DE SOFTWARE (ENFOQUE TRADICIONAL)

La estimación es un proceso probabilístico que busca predecir aspectos del proyecto antes de que ocurran, como tamaño, esfuerzo, tiempo y costo. Es importante entender que estas estimaciones se realizan con información limitada y siempre tienen un grado de incertidumbre.

Tenemos que tener en cuenta que:

- La estimación nunca es precisa
- Estimar no es planear y planear no es estimar
- Las estimaciones son la base de los planes, pero el plan no tiene que coincidir con la estimación
- A mayor diferencia entre lo estimado y lo planificado, mayor es el riesgo: si el plan no respeta demasiado la estimación, el proyecto corre más incertidumbre.
- Las estimaciones no son compromisos: no deben tomarse como una promesa rígida, sino como una guía para tomar decisiones y gestionar riesgos

MÉTODOS UTILIZADOS EN EL ENFOQUE TRADICIONAL

1. Métodos basados en la experiencia

- **Datos históricos** → se recopila información de proyectos anteriores y se utilizan como referencia para estimar nuevos proyectos. Se tiene que elegir que datos son los que se van a tomar como referencia.
- **Juicio experto Puro** → un experto analiza las especificaciones y realiza la estimación basándose en su conocimiento. Tiene la desventaja de que, si el experto no está disponible, se pierde esa capacidad de estimar.
- **Juicio Experto Delphi** → en lugar de depender de un solo experto, se recurre a un grupo de expertos que aportan sus estimaciones y se busca un consenso.
- **Analogía** → se compara el proyecto actual con otros similares y se estima a partir de esas similitudes.

2. Basados exclusivamente en los recursos.

3. Método basado exclusivamente en el mercado.

4. Basados en los componentes del producto o en el proceso de desarrollo.

5. Métodos algorítmicos.

ERRORES EN LAS ESTIMACIONES

Una de las fuentes más comunes de error en las estimaciones es omitir actividades necesarias para completar correctamente el proyecto. Algunos ejemplos:

- Requerimientos faltantes: no se contemplan todas las necesidades del sistema.
- Actividades de desarrollo omitidas: documentación técnica, revisiones, preparación de datos para testing, mantenimiento de versiones previas, etc.
- Actividades generales: imprevistos como licencias, capacitaciones, reuniones internas o ausencias por enfermedad.

ESTIMACIONES EN ENFOQUE ÁGIL

En Agile, lo más valioso no es la estimación en sí, sino lo que se aprende durante el proceso de estimar. Se basa en la idea de que los seres humanos somos mejores haciendo estimaciones relativas que absolutas. Es decir, nos resulta más fácil comparar dos tareas que calcular con precisión cuánto tiempo llevará una sola.

Las estimaciones no se hacen todas al inicio, ya que al principio del proyecto hay mayor incertidumbre. En lugar de eso, se aplican de forma Just in Time, es decir, se estiman las User Stories a medida que se necesitan, reduciendo el riesgo de errores.

Los Story Points son la unidad de medida utilizada para expresar el tamaño relativo de una historia de usuario, se la mide en base a las siguientes 3 dimensiones:

- Complejidad
- Esfuerzo
- Incertidumbre

POKER ESTIMATION

Es un método de estimación ágil que combina:

- Opinión de experto
- Analogía (comparar con otras historias conocidas)
- Descomposición (dividir en partes más pequeñas si es necesario)

Los desarrolladores son quienes realizan las estimaciones → “Las personas más competentes en resolver una tarea deben ser quienes las estiman”

En este método los desarrolladores asignan valores a las User Stories de forma colaborativa y relativa. Cada miembro del equipo tiene un mazo de cartas con valores de la serie de Fibonacci y, tras discutir la historia con el Product Owner, todos eligen una carta al mismo tiempo. Si los valores coinciden, se adopta ese número; si no, se debaten las diferencias y se vuelve a votar hasta alcanzar consenso. El proceso se repite con todas las historias, siempre comparándolas con una historia base de referencia.

GESTIÓN DEL PRODUCTO

En el desarrollo de software suele invertirse mucho esfuerzo en funcionalidades que rara vez se usan. Por eso, la gestión de producto busca eliminar desperdicio y enfocarse en entregar valor real al usuario. El éxito no es entregar un producto, el éxito se trata de entregar un producto (o característica de producto) que el cliente usará.

Para lograrlo se usan distintos conceptos, que representan distintos niveles de granularidad:

- **MVP (Producto Mínimo Viable):** es la primera versión mínima del producto que permite validar si la idea realmente interesa a los clientes (hipótesis inicial). Si el feedback es positivo, se sigue construyendo; si no, se pivotea hacia una nueva hipótesis. Está dirigido a un subconjunto de clientes potenciales.
- **MVF (Funcionalidad Mínima Viable):** se utiliza cuando hay incertidumbre sobre una funcionalidad en particular. Es como un “mini-MVP” enfocado en una característica específica para validar si realmente genera valor antes de invertir más recursos en ella.
- **MMF (Funcionalidad Mínima Comercializable):** es una funcionalidad pequeña pero completa, que ya se puede entregar a los usuarios y que aporta un valor real tanto para ellos como para el negocio.
- **MMR (Release Mínimo Comercializable):** Es un conjunto de funcionalidades (MMF) integradas en un release que ya entregan un valor completo al usuario.

- **MRF (Funcionalidad Mínima del Release):** es la característica mínima necesaria que debe incluir un release para que tenga sentido y aporte valor al usuario. Es el conjunto de MMF que se lanzan en un release porque tienen sentido juntos.
- **MMP (Producto Mínimo Comercializable):** Es el primer MMR (MMP = MMR1). Está enfocado en un grupo especial de usuarios: los early adopters (los primeros en probar algo nuevo). El MMP tiene solo las funcionalidades clave necesarias para que este grupo inicial lo adopte y lo use, y a partir de su feedback se van construyendo los siguientes MMR.



MÉTODO LEAN STARTUP

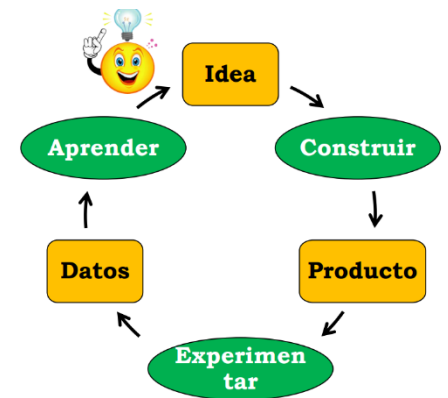
El Lean Startup es un enfoque para crear productos o negocios de forma rápida y eficiente, reduciendo riesgos y desperdicio. Su idea principal es probar hipótesis antes de invertir mucho tiempo o dinero en el producto completo.

LA AUDACIA CERO

Es un fenómeno que ocurre especialmente en startups y desarrollo de nuevos productos. Se refiere a la idea de que es más fácil conseguir financiamiento cuando el proyecto todavía no tiene ingresos, clientes ni tracción, es decir, cuando todo está “en cero”.

Esa “audacia del cero” empuja a muchos a retrasar el lanzamiento hasta estar 100% seguros del éxito. La forma de contrarrestar la “audacia de cero” es lanzar un MVP temprano:

- Probar la idea con usuarios iniciales o early adopters.
- Concentrarse en los elementos más riesgosos primero, aquellos que podrían invalidar la hipótesis si fallan.
- Obtener feedback real antes de invertir demasiado tiempo o recursos.

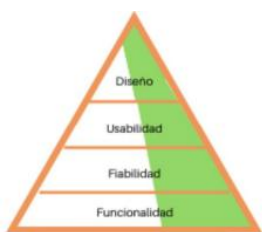


SUPUESTOS DE SALTOS DE FE

Los supuestos de “saltos de fe” son ideas o hipótesis que se dan por ciertas al iniciar un proyecto, pero que aún no se han validado. En metodologías ágiles, estos supuestos se identifican para poder probarlos rápidamente con un MVP u otros experimentos, de modo que se reduzca el riesgo de construir algo que no genere valor real.

- Hipótesis del valor → Prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo.
- Hipótesis de crecimiento → Prueba cómo nuevos clientes descubrirán el producto

CARACTERÍSTICAS DE UN MVP



Cómo SÍ hacer un MVP



Cómo NO hacer un MVP

Diseño	• Diseño adecuado. • Consigue una UX que deleita. • Logra satisfacer el aspecto visual y de interacción.
Usabilidad	• Tiene suficiente valor para que la gente esté dispuesta a usarlo/comprarlo. • Resulta útil para su público objetivo.
Confiable	• Involucra a los early adopters puedan confiar en la solución plenamente. • Incluso cuando tenga poco tiempo en el mercado.
Funcionalidad	• Tiene las funciones necesarias para solucionar un problema específico. • Satisface las demandas y permite evaluar las funciones a implementar más adelante.

SCRUM

SCRUM es un framework ágil liviano que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas frente a problemas complejos.

Adhiere a los 3 pilares del empirismo: Transparencia, inspección y adaptación, y utiliza un enfoque iterativo e incremental entregando valor en pequeños incrementos y mejorando el producto de manera continua a lo largo del proyecto.

EQUIPO DE SCRUM

- Es una unidad cohesionada de profesionales que trabaja de manera conjunta hacia un único objetivo: el éxito del producto.
- Es un equipo multifuncional, lo que significa que sus miembros poseen todas las habilidades necesarias para generar valor en cada sprint.
- Es autogestionado, por lo que los integrantes deciden internamente quién hace qué, cuándo y cómo, sin depender de una autoridad externa.
- Está compuesto por 10 personas o menos, lo que facilita la comunicación y la colaboración efectiva.

RESPONSABILIDADES

- Desarrolladores → encargados de crear el Incremento funcional en cada Sprint. Se responsabilizan de: planificar el Sprint (Sprint Backlog), garantizar la calidad (DoD), adaptar diariamente su trabajo al Objetivo del Sprint y apoyarse mutuamente como profesionales.
- Product Owner → representa el lado del negocio dentro de Scrum y es responsable de maximizar el valor del producto que el equipo desarrolla. Gestiona el Product Backlog, comunica el Objetivo del Producto y prioriza el trabajo según el valor de negocio.
- Scrum Master → garantiza que Scrum se entienda y se aplique correctamente, mejorando la efectividad del equipo.

EVENTOS

Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum. Si no se realizan los eventos según lo prescrito, se pierden oportunidades para inspeccionar y adaptarse.

SPRINT

Durante el Sprint se realiza todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la planificación del Sprint (Sprint Planning), las reuniones diarias (Daily Scrum), la revisión del Sprint (Sprint Review) y la retrospectiva (Sprint Retrospective). Un Sprint podría ser cancelado si el Objetivo del Sprint se vuelve obsoleto. Solo el PO tiene la autoridad para cancelar el Sprint.

SPRINT PLANNING

La planificación del Sprint es la reunión que inicia cada Sprint y define qué trabajo se hará y cómo. Todo el equipo de Scrum colabora para crear este plan. Durante la planificación se responden tres preguntas clave:

1. ¿Por qué este Sprint es valioso? el PO propone qué se quiere lograr para aportar valor al usuario o al negocio, y el equipo define el Objetivo del Sprint (Sprint Goal), que resume en una frase clara por qué el Sprint es importante.
2. ¿Qué se puede hacer en este Sprint? Los desarrolladores, junto con el Product Owner, seleccionan los elementos del Product Backlog que se completarán en el Sprint.
3. ¿Cómo se realizará el trabajo elegido? Cada elemento del Sprint se descompone en tareas pequeñas que el equipo puede completar en uno o pocos días. Los desarrolladores deciden cómo ejecutar el trabajo.

DAILY SCRUM

Es una reunión diaria de máximo 15 minutos donde los desarrolladores del equipo revisan el progreso hacia el Objetivo del Sprint y ajustan el plan de trabajo (Sprint Backlog) si es necesario. Se realiza siempre en el mismo lugar y hora para mantener consistencia.

SPRINT REVIEW

Es una reunión donde el equipo de Scrum presenta lo que logró durante el Sprint a las partes interesadas y se analiza el progreso hacia el Objetivo del Producto. Se revisa lo que se completó, lo que ha cambiado en el entorno y se decide qué hacer a continuación.

SPRINT RETROSPECTIVE

Es la reunión final del Sprint, donde el equipo de Scrum analiza cómo fue el último Sprint y busca formas de mejorar su trabajo. Se revisan aspectos como interacciones, procesos, herramientas y la definición de “Hecho”, identificando qué funcionó bien, qué problemas surgieron y cómo resolverlos.

REFINAMIENTO DEL PRODUCT BACKLOG

Es un evento en el que el equipo de Scrum revisa y mejora los ítems del Product Backlog. Se aclaran detalles, se priorizan, se estiman o re-estiman y se identifican riesgos o dudas. El objetivo es que los ítems estén listos para entrar a un Sprint cumpliendo la Definition of Ready (DoR).



ARTEFACTOS

Los artefactos de Scrum representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Cada artefacto tiene un compromiso que ayuda a medir el progreso:

- Product Backlog: Compromiso → Product Goal
- Sprint Backlog: Compromiso → Sprint Goal
- Incremento: Compromiso → Definition of Done

PRODUCT BACKLOG

El Product Backlog es la lista ordenada de todo lo necesario para mejorar el producto y es la única fuente de trabajo que el equipo de Scrum toma para sus Sprints. Sus elementos se refinan continuamente, agregando detalles, descripciones y tamaños estimados. Los desarrolladores son responsables de estimar el tamaño, mientras que el Product Owner guía la priorización y ayuda a tomar las mejores decisiones.

Compromiso: Product Goal → El Product Goal describe un estado futuro del producto que sirve como objetivo a largo plazo. Representa el valor que se busca entregar.

SPRINT BACKLOG

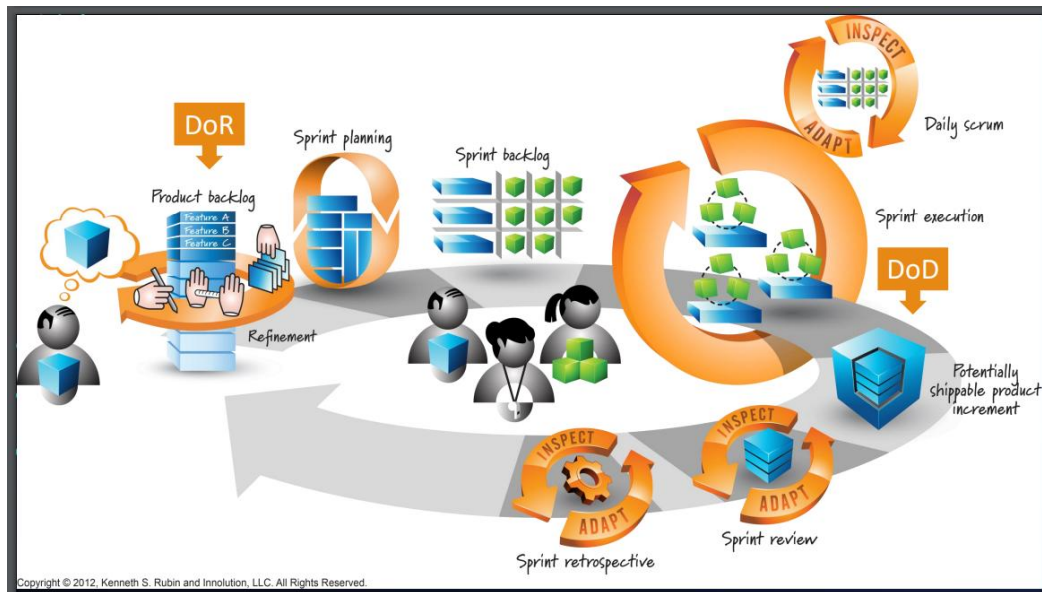
El Sprint Backlog es el conjunto de trabajo que el equipo de Scrum se compromete a completar durante un Sprint. Contiene tres elementos clave: el Sprint Goal (el “por qué” del Sprint), los elementos del Product Backlog seleccionados (el “qué” se va a hacer) y el plan de acción (el “cómo” se entregará el incremento). Es un plan hecho por y para los desarrolladores, que se actualiza continuamente durante el Sprint.

Compromiso: Sprint Goal → El Sprint Goal define un objetivo claro para el Sprint y mantiene al equipo enfocado en entregar valor, aunque los detalles del trabajo puedan cambiar durante el Sprint

INCREMENTO

Un Incremento es un paso concreto hacia el Product Goal y se suma a los incrementos anteriores, formando una versión funcional y verificada del producto que aporta valor utilizable.

Compromiso: Definition of Done (Definición de Hecho) → La Definición de Hecho es la descripción formal de cuándo un incremento se considera completo según los estándares de calidad del producto. Solo se considera Incremento aquello que cumple esta definición, garantizando que todos tengan una comprensión común de lo que significa “terminado”.



¿QUÉ SE ESTIMA?

En Scrum se hacen estimaciones para poder planificar y priorizar el trabajo, pero no son exactas, sino relativas y adaptativas. Se estima en diferentes niveles:

- Portfolio Backlog
- Product Backlog
- Sprint Backlog

PLANIFICACIÓN DEL RELEASE

Un release (o versión) es básicamente una entrega del producto a los usuarios o clientes. No necesariamente significa que sea la versión final, sino que es una entrega funcional que aporta valor, aunque solo incluya algunas funcionalidades.

Con base en las estimaciones del Product Backlog, se define qué funcionalidades se entregarán en cada release, cuándo se entregarán y cómo aportarán valor al usuario. Permite organizar entregas parciales y minimizar riesgos, alineando lo que se construye con lo que realmente importa al negocio.



MÉTRICA DE SCRUM

En Scrum, la métrica principal es la velocidad, que mide software funcionando entregado por el equipo durante un Sprint. Se calcula sumando los Story Points (SP) de todas las User Stories completadas y aceptadas por el Product Owner. Las US que no cumplen con la Definition of Done (DoD) o son rechazadas por el PO no se incluyen en la velocidad y regresan al Product Backlog.